

Gazelle 光子计算评估板

产品手册

文件编号			
文件状态	正式发布		
当前版本	V1.0.1		
拟 制	杨春	日期	2026/6/20
审 核	杨春, 冯 晓敏	日期	2026/6/20
批 准	冯晓敏	日期	2026/6/20

目录

1.产品概述	3
1.1 评估板简介	3
1.2 硬件框图	3
2.技术参数	5
2.1 硬件参数	5
2.1.1 评估板参数	5
2.1.2 激光器参数	6
2.1.3 光子计算系统参数	7
2.2 软件参数	7
3.快速上手	8
3.1 连接开发板	8
3.1.1 串口线连接	8
3.1.1.1 串口驱动安装	8
3.1.1.2 工具软件安装	8
3.1.1.3 客户端 PC 与 Gazelle 单板连接	8
3.1.1.4 固定 Gazelle ip(选做)	10
3.1.2 SSH 连接	11
3.2 Gazelle API 调用运行测试	11
3.2.1 用户登录	11
4.编程接口和校准	13
4.1 软件说明	13



4.2 软件接口说明	14
4.3 Gazelle 评估与校准	16
4.3.1 登录.....	16
4.3.2 精度评估.....	16
4.3.3 校准.....	17
5.实际样例	19
Compass 提供本地和远程两种方式实现服务,	19
5.1 矩阵点乘（本地执行）	19
5.2 矩阵点乘（远程）	19
6.FAQ	22

1. 产品概述

1.1 评估板简介

Gazelle 光子计算评估板是一款基于光子学原理的新一代光子计算研究平台，支持 8x2 基础光子矩阵计算。该评估板的光子计算芯片具有两大优势：第一是可编程性，用户可以通过 API 配置光子计算矩阵的系数，从而运行更多的定制算法；第二是高精度，输入数据位宽 8bits，输出 EBR（Effective Bit Resolution）大于等于 8，可以支持高精度的矩阵向量乘法。此外，该产品可通过板载 FPGA 与光子芯片融合实现多种计算负载。该产品支持多种常用调试接口，板载高性能 AD、DA、TIA 芯片，配有外置激光器。用户可通过对 FPGA 编程实现对计算负载的软件/硬件间分配和电计算/光计算间分配。

Gazelle 评估板可以用来求解 QUBO 这一大类问题，比如伊辛问题、max-cut 问题、SAT 问题等；也可用于神经网络模型的推理，可以使其运用到图像识别、目标检测等人工智能场景中。

该产品可广泛应用于人工智能、量子计算、网络安全等领域，为这些领域的科研和工业生产提供强有力的支持。同时，该产品还可以解决传统计算面临的瓶颈问题，具有广阔的市场前景和应用价值。



图 1. Gazelle 光子评估板图

1.2 硬件框图

整个 Gazelle 计算系统的硬件由 PIC，激光器，FPGA 等组成，用户通过 FPGA 接口控制光子计算芯片完成计算任



务。

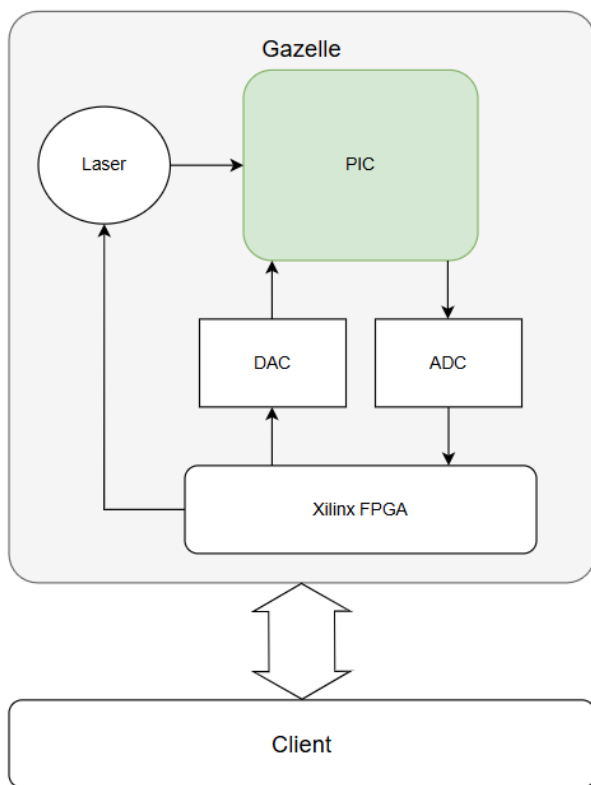


图 2. Gazelle 系统框架图



2.技术参数

2.1 硬件参数

2.1.1 评估板参数

组件	规格型号	数量
光子芯片	Gazelle	1
SOM 板	米联客	1
电源适配器	12V@5A	1
Socket	LGA50x50-M47x47-1.00-488, V2 ¹	1
FPGA	Zynq MP	1
内存规格	PS 端：4GB DDR4, PL 端：1GB DDR4,	1
光子芯片数据格式及算力	数据格式： ▪ 输入格式(向量): uint8 ▪ 输入格式(矩阵)：int8 ▪ 输出 EBR: 大于等于 8² 算力：2.6MOPs ³	
接口	规格型号	数量
网口	1000Mbps	1
USB 2.0	USB host	1
SD	SD 卡接口 (卡容量不小于 32GB)	1
供电接口	12V@5A	2
电气和结构参数		
功耗	20 W	
工作环境温度	25~35 ° C	
结构尺寸	380mm x 320mm x 80mm	

表 1. Gazelle 光子计算评估板参数

- 1.该 socket 为曦智科技特供型号
2. EBR of operator(effective bit resolution, 即有效比特分辨率)定义如下：

$$EBR = \log_2 \left(\frac{DR}{std(error)} \right)$$

$$error = H \cdot S - T$$

其中

T=True results

H=Hardware outputs

S=Scaling factor

DR=True value range, $2^{n1+n2+\log_2(n3)} - 1$

n1 is the vector bit width



n2 is the weight/Matrix elements bit width

n3 is the size of dot-product

3. 矩阵规模 (1024, 1024) * (1024, 1024), adddelay 1000 下。

4. 仅 EVB 功耗, 不含激光器

2.1.2 激光器参数

Moonstone 单波长激光光源是一款广泛应用的小型化、高功率、单波长的激光光源, 该激光光源的光耦合效率和控温设计良好, 同时具备高功率和低功耗的性能优点。该激光光源内部光组件采用半气密封装, 具备良好的散热设计, 通道独立控制, 支持 RS485 通讯, 标准 MTP 光接口。

参数	典型规格参数
储存环境温度	-40~85°C
工作环境温度	0~55°C

表 2. Moonstone 激光光源环境温度表

参数	性能参数
电源	12V 3A 电源适配器
功耗	~11W(室温, ILD=400mA, TLD=50°C)
温度调节范围	35~65°C, 典型值 50°C
中心波长	1310nm

表 3. Moonstone 激光光源电学特性表

2.1.3 光子计算系统参数

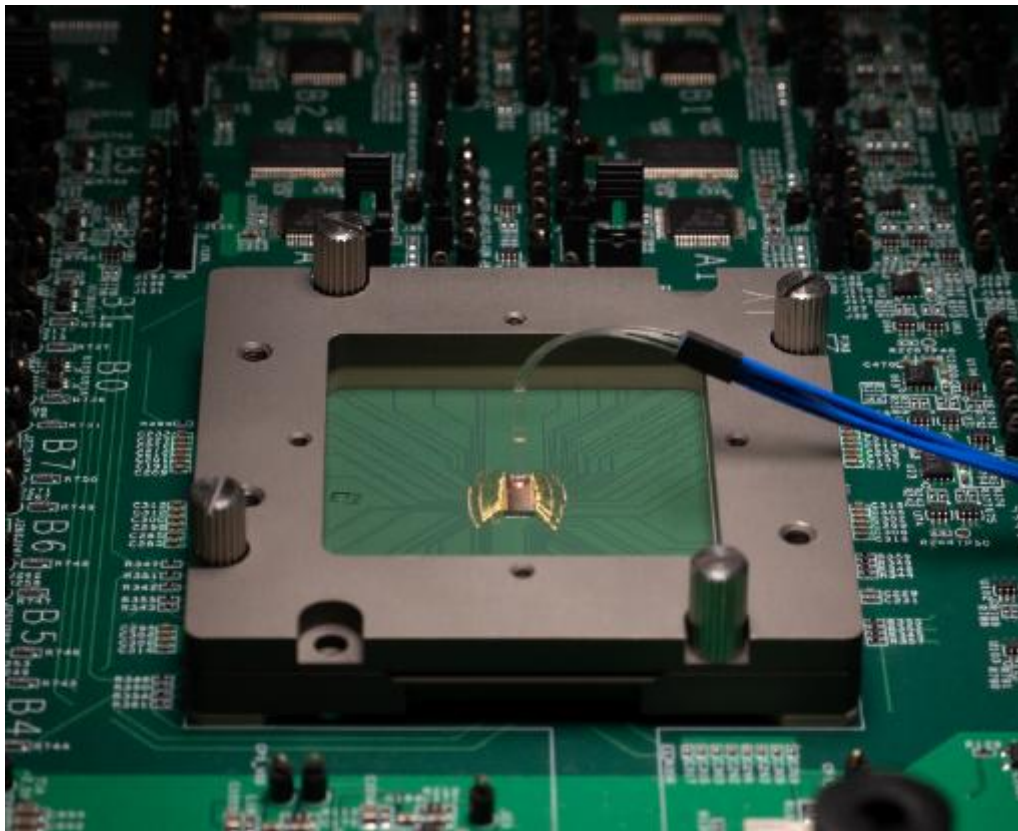


图 3. PIC 实景图

参数	性能参数
输入 1	▪ 输入范围：0~255 (uint8)
输入 2	▪ 输入范围：-128 ~127 (int8)
输出	▪ 输出 EBR： 大于等于 8

表 4. PIC 参数

2.2 软件参数

Gazelle 评估板的嵌入式软件 PL 部分主要完成 AD、DA 的数据采集，并且接收 ps 部分的命令，解析并执行相关命令。

Gazelle 评估板的嵌入式软件 PS 部分使用 linux 系统，主要完成电路板各个功能模块的驱动、测试逻辑、数据通信交互部分的功能。

Gazelle 评估板的应用开发，主要通过片上自带的 python 库 compass_sdk 实现。



3.快速上手

3.1 连接开发板

为在 Gazelle 评估板上顺利进行光子计算 API 的使用，需要准备一台 PC 电脑，并接好 Gazelle 电源线，串口线及网线。

PC 与 Gazelle 评估板有两种连接方式：1.通过串口线直接连接；2.通过网线将 Gazelle 接入局域网并获取其 IP 地址，PC 通过 ssh 形式连接。

3.1.1 串口线连接

3.1.1.1 串口驱动安装

在 windows 系统下，需要安装一个串口转 USB 的驱动，才能正常使用该接口。安装驱动后，windows 系统会将该接口识别为两个串口。其中有一个串口为电路板的调试串口，即电路板嵌入式 linux 系统的终端，可执行终端 shell 命令等功能。

推荐使用驱动为 CP210x_Windows_Win10_Drivers.rar。

3.1.1.2 工具软件安装

工具软件的功能主要为连接调试端口，作为终端使用；推荐安装 PuTTY 工具，或者使用功能类似的软件。

3.1.1.3 客户端 PC 与 Gazelle 单板连接

- 准备一根 USB 连接线，一端连接到 Gazelle_evb 的 microUSB 口，一端连接到电脑的 USB 接口。
- 如果 PC 端串口驱动已经安装，可以从我的电脑-->设备管理器中看到端口一栏下面有两个 COM 端口（具体编号由系统自动生成配置）。
- 打开 PuTTY，新建会话（由于 Gazelle 评估板会自动选择通讯端口，请在设备管理器中确认实际使用的 COM 端口号，并在 PuTTY 中对应设置）。
- 选择协议为 Serial，然后波特率选择 115200，设置后，点击 open 建立连接。

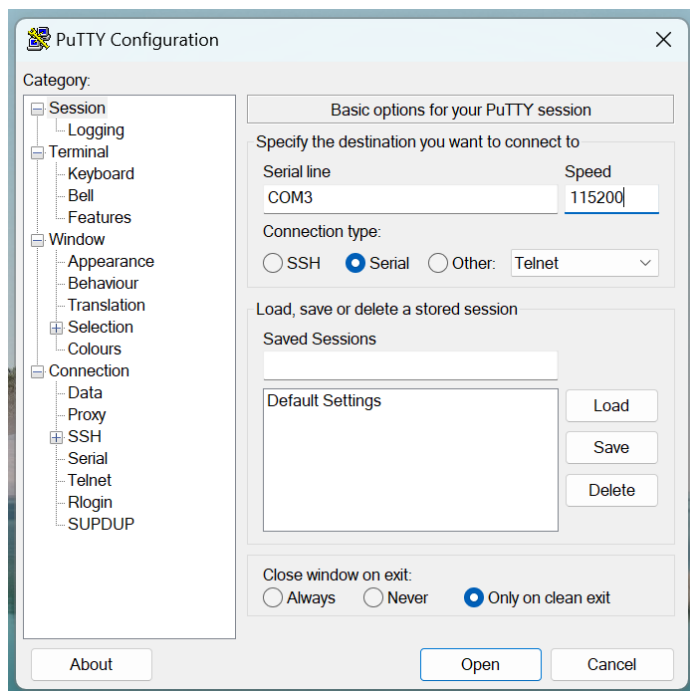


图 4. 连接参数示意图

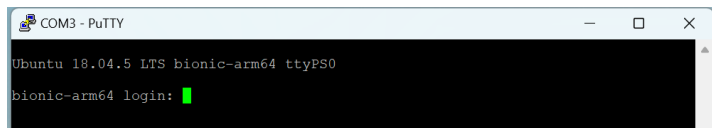


图 5. 连接成功输入用户名、密码示意图

在 PuTTY 连接终端，输入用户名：uisrc，密码：root。如果连接成功则无需再尝试另外一个 COM 口。如果连接失败，说明 Gazelle 评估板是使用的另一个端口，使用同样的方法创建连接，注意端口选择。下图连接成功显示内容：

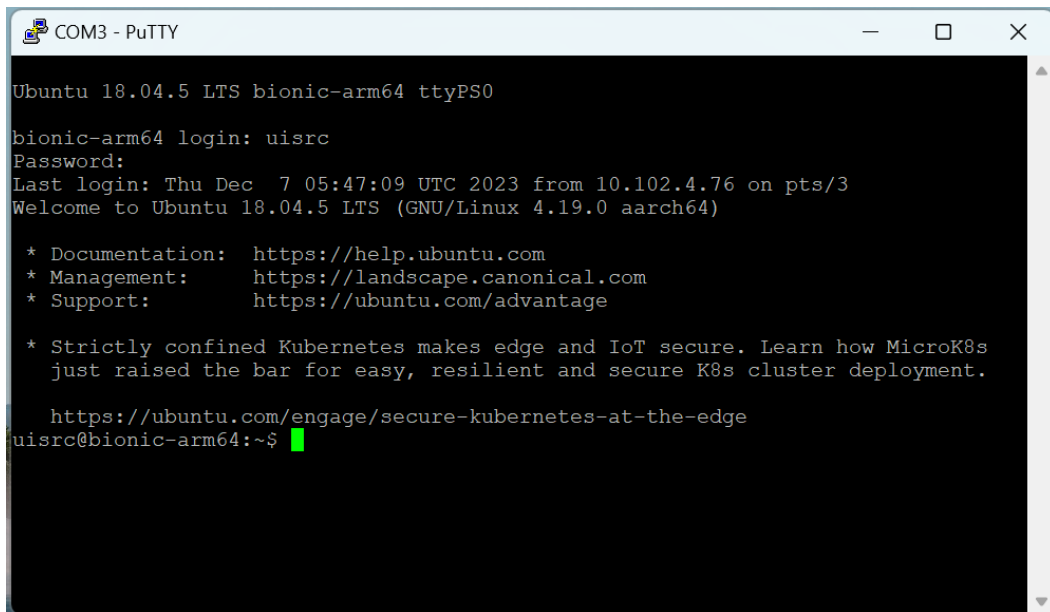


图 6 登陆成功示意图



若出现如上界面，则表示 Gazelle 评估板连接成功。此时会自动进入操作系统的根目录，可以通过 ifconfig 命令查看系统的 IP（开发板默认 IP 地址为 DNS 自动分配，每次启动可能不同）。

```
uisrc@bionic-arm64:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.102.1.82 netmask 255.255.255.0 broadcast 10.102.1.255
    inet6 fe80::7f9c:1645:b385:3f4b prefixlen 64 scopeid 0x20<link>
    ether 06:83:fa:2e:80:8e txqueuelen 1000 (Ethernet)
    RX packets 297074 bytes 37767480 (37.7 MB)
    RX errors 3 dropped 0 overruns 0 frame 3
    TX packets 109250 bytes 16711473 (16.7 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 30

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 244994 bytes 20436529 (20.4 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 244994 bytes 20436529 (20.4 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

图 7. 评估板获取到的 IP 地址

备注：获取到 Gazelle 评估板的 IP 后，可以采用 terminal 通过 ssh 的形式进行访问。

3.1.1.4 固定 Gazelle ip(选做)

如果需要设置成静态固定的 IP+ssh 登陆的方式，采用如下方法进行设置：

第一步：切换 Root 权限，进入对应目录下修改网卡配置文件。

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Last login: Mon Dec 11 02:56:51 2023
uisrc@bionic-arm64:~$ su
root@bionic-arm64:/home/uisrc# cd /etc/network
root@bionic-arm64:/etc/network# ls
if-down.d if-post-down.d if-pre-up.d if-up.d interfaces interfaces.d
root@bionic-arm64:/etc/network# vi interfaces
root@bionic-arm64:/etc/network#
```

切换为root权限

进入文件夹

打开编辑该文件，配置静态IP

图 8.网卡配置文件示意图

第二步：参考如下字段的配置修改配置文件。

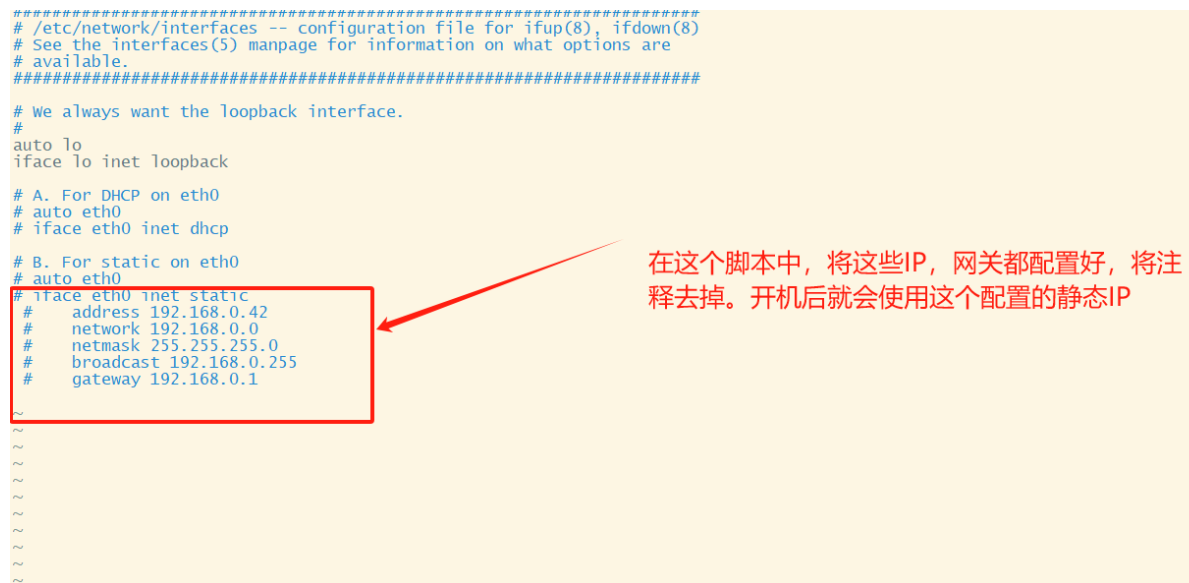


图 9. 修改字段示意图

第三步：断电重启后 Gazelle 就可以实现固定 IP 登陆。

3.1.2 SSH 连接

知道 gazelle 的 ip 地址后，用户可以通过 ssh 方式登录 gazelle，使用 sftp 或 scp 进行数据传输。

3.2 Gazelle API 调用运行测试

3.2.1 用户登录

Gazelle 支持两种方式登录

1. 利用 PC 通过串口登录 Gazeela，见 3.2.2。默认登录名 uisrc，密码 root。
2. 利用网络以 ssh 方式登陆 Gazelle，默认登录名 root，密码 root。

第一步：用户登录

（方法 1）采用串口形式登录：

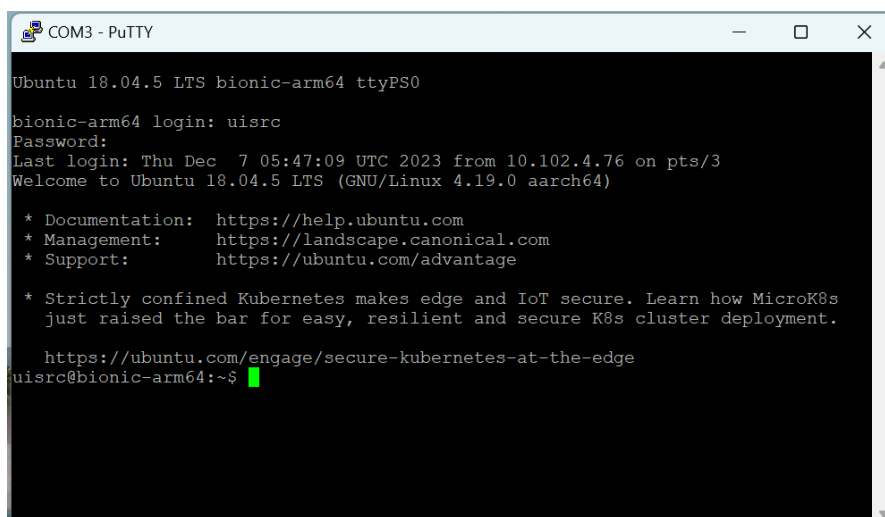


图 10. 串口登陆示意图

(方法 2) 采用 ip 地址登录：

```
Connection to 10.102.1.82 closed.
(base) yanghuang@yanghuangdeMacBook-Air ~ % ssh uisrc@10.102.1.82
uisrc@10.102.1.82's password:
Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.19.0 aarch64)
```

图 11. IP 登陆示意图

第二步：切换账户

运行代码需要在 root 用户下运行，如果是其他用户登录，可输入命令 su 切换到 root 用户，密码为 root，然后再运行。

第三步：运行测试

输入命令 “compass_evb_test”会执行测试程序。

```
root@bionic-arm64:/home/uisrc# compass_evb_test
open /dev/ttyPS1 serial error
open file sucess
set chaneel1 laster param: /dev/ttyUSB0 is y = 0.2447x - 5.4282
open file sucess
%I:rhs: file_name_LUT_typ:doublearm
set ch1 current 83.48mA
```

图 12.精度测试程序

4.编程接口和校准

4.1 软件说明

软件 SDK 默认安装在 Gazelle 中 root 账户下，

```
root@bionic-arm64:/home/uisrc# pip3 list|grep compass
/usr/local/lib/python3.6/dist-packages/secretstorage/dhcr
rt for it is deprecated in cryptography. The next release
from cryptography.utils import int_from_bytes
compass-sdk 1.0.2
```

图 13. SDK

compass_sdk 提供 compass_init, compass_matmul, compass_matmul_advance 等接口，用户可以在 Python 程序中调用接口完成光计算。

在 “/home/uisrc/code” 中提供相应的示例代码，用户可参考其中代码编写程序。

```
calibrate_sample.py
compass_client.py
compass_server.py
evb_test_sample.py
local_matmul_advance_sample.py
local_matmul_sample.py
remote_matmul_sample.py
```

图 14.示例代码列表

- compass_client.py 为运行远程 matmul 服务的客户端，需运行于远程机器上
- compass_server.py 为运行远程 matmulfu 服务的服务端，运行于 Gazelle 本地
- calibrate_sample.py 代码方式调用校准
- evb_test_sample.py 代码方式验证精度
- local_matmul_sample.py 本地调用 matmul 代码示例
- local_matmul_advance_sample.py 本地调用 matmul_advance 代码示例
- remote_matmul_sample.py 远程调用 matmul 示例

4.2 软件接口说明

软件栈提供几个 API 接口，供用户调用，主要定义于 `compass_sdk.fast_calibration.compass_lib` 库中：

`compass_init()`

功能：初始化 Gazelle，本地调用时启动程序时调用

参数：无

`compass_matmul(vec, weight)`

功能：本地调用光计算芯片实现矩阵计算

参数：

- Vec: 输入数据
 - ◆ 输入类型, `numpy.ndarray`,
 - ◆ 数据类型, `numpy.uint8`,
- Weight: 输入权重
 - ◆ 输入类型, `numpy.ndarray`,
 - ◆ 数据类型, `numpy.int8`

限制：输入数据规模不超过(1024x4096), 输入权重规模不超过(4096x1024), 输出矩阵规模不超过(1024x1024)

`compass_matmul_advance(vec, weight)`

功能：本地调用光计算芯片实现矩阵计算，针对矩阵中数值较小的数据，进行了 auto scaling, 实现计算精度优化

参数：

- Vec: 输入数据
 - ◆ 输入类型, `numpy.ndarray`,
 - ◆ 数据类型, `numpy.uint8`,
- Weight: 输入权重
 - ◆ 输入类型, `numpy.ndarray`,



- ◆ 数据类型, numpy.int8

compass_matmul(vec, weight, ip, port)

功能： 远程调用光计算芯片实现矩阵计算

参数：

- Vec: 输入数据
 - ◆ 输入类型, numpy.ndarray
 - ◆ 数据类型, numpy.uint8,
- Weight : 输入权重
 - ◆ 输入类型, numpy.ndarray,
 - ◆ 数据类型, numpy.int8
- Ip: Gazelle ip 地址
- Port: 服务端口, 默认为 8000, 可通过启动 compass_server.py 后查看 log 观察端口号

限制： 输入数据规模不超过(1024x4096), 输入权重规模不超过(4096x1024), 输出矩阵规模不超过(1024x1024)

compass_matmul_advance(vec, weight, ip, port)

功能： 远程调用光计算芯片实现 auto scaling 矩阵计算

参数：

- Vec: 输入数据
 - ◆ 输入类型, numpy.ndarray
 - ◆ 数据类型, numpy.uint8,
- Weight : 输入权重
 - ◆ 输入类型, numpy.ndarray,
 - ◆ 数据类型, numpy.int8
- Ip: Gazelle ip 地址
- Port: 服务端口, 默认为 8000, 可通过启动 compass_server.py 后查看 log 观察端口号

compass_calibrate()

功能： 校准程序, 用于对光计算芯片进行校准。



参数：无

compass_evb_test()

功能：性能测试程序，利用光计算芯片计算 10000 组固定的矩阵乘法数据，利用统计值评估光计算芯片的精度

参数：无

4.3 Gazelle 评估与校准

如果 Gazelle 评估板的硬件误差超出容许范围，可以通过校准的方式降低误差，具体步骤如下：

4.3.1 登录

- 通过串口或者网络方式登录 Gazelle
- 若非 root 账户，切换到 root 账户

4.3.2 精度评估

检测当前性能，如果已安装 SDK，则可以通过命令方式直接调用测试脚本，若未安装，请安装软件包。用户可以选

择任一方式进行性能评估。

- 命令行

在 root 账户下，执行 compass_evb_test

```
root@bionic-arm64:~# compass_evb_test
open /dev/ttyPS1 serial error
open file sucess
set chaneel1 laster param: /dev/ttyUSB0 is  $y = 0.2447x - 5.4282$ 
open file sucess
%I:rhs: file_name_LUT_typ:doublearm
set ch1 current 83.48mA
```

图 15. 精度评估命令

- 程序

切换到 “/home/uisrc/code/samples” 路径，执行 “python3 evb_test_sample.py”

程序执行日志如下



```
root@bionic-arm64:/home/uisrc# compass_evb_test
open /dev/ttyPS1 serial error
open file sucess
set chaneel1 laster param: /dev/ttyUSB0 is y = 0.2447x - 5.4282
open file sucess
%I:rhs: file_name_LUT_typ:doublearm
set ch1 current 83.48mA
open file sucess
%I:rhs: get die_type is die4
%I:rhs: get die_id is N4-D09-1-EVB02-2mA(new)
open file sucess
open file sucess
%I:rhs: get die_type is die4
%I:rhs: get die_id is N4-D09-1-EVB02-2mA(new)
open file sucess
%I:rhs: get vdd is 1.8
%I:rhs: get vee is 0.0
set vee = 0.0,set io value =0
set vdd = 1.8
ltc2264 reg read, data=[255, 135]
ltc2265 verify communication failed,read reg value is:135
ltc2264 reg read, data=[255, 128]
data is :0x80
ltc2264 reg read, data=[255, 203]
data is :0xcb
%I:rhs: get Power_ref =1000, get gc_mpd_offset=2045
read finished flag =3, before update LUT
read finished flag =1, count = 0
read finished flag =3, count = 1
%I:rhs: config tx calibration value: sw_ch = [9, 10, 11, 12, 13, 14, 15, 16],sw_val=[0, 0, 0, 0, 1, 0, 1, 0, 1, 0], dac_ch = [9, 10, 11, 12, 13, 14, 15, 16], dac_val=[251, 0, 910, 1402, 3549, 973, 3744, 2095]
open file sucess
%I:rhs: get die_type is die4
%I:rhs: get die_id is N4-D09-1-EVB02-2mA(new)
%I:rhs: mg_wgh1_sw_list is [17, 18, 19, 20, 21, 22, 23, 24]
%I:rhs: mg_wgh2_sw_list is [25, 26, 27, 28, 29, 30, 31, 32]
open file sucess
%I:rhs: get offset value is [2051.29, 2045.07]
```

图 16. 精度评估程序日志

4.3.3 校准

如果测试精度小于预期，需要执行校准程序，对硬件进行校准。SDK 提供两种方式进行校准。

- 命令行

在 root 账户下，执行 compass_cali 命令，会启动校准程序。

- 程序

切换到 “/home/uisrc/code/samples” 路径，执行 “python3 calibrate_sample.py”。

程序执行日志如下：



```
open file success
%lrhs: starting calibration_light_mzi
open file success
%lrhs: get die_type is die4
%lrhs: get die_id is N4-D09-1-EVB02-2mA(new)
open file success
%lrhs: get die_type is die4
%lrhs: get die_id is N4-D09-1-EVB02-2mA(new)
open file success
%lrhs: starting calibration_light_mzi sequence
open file success
%lrhs: get die_type is die4
%lrhs: get die_id is N4-D09-1-EVB02-2mA(new)
%lrhs: config tx calibration value: sw_ch = [9, 10, 11, 12, 13, 14, 15, 16], sw_val=[0 0 0 0 1 0 1 0], dac_ch = [9, 10, 11, 12, 13, 14, 15, 16], dac_val=[ 251   0 910 1402 3549 97
3 3744 2095]
%lrhs: open file success
open file success
%lrhs: get vdd is 1.8
%lrhs: get vee is 0.0
open file success
%lrhs: get die_type is die4
%lrhs: get die_id is N4-D09-1-EVB02-2mA(new)
open file success
open file success
%lrhs: get die_type is die4
%lrhs: get die_id is N4-D09-1-EVB02-2mA(new)
%lrhs: starting weight data collection
%lrhs: config tx calibration value: sw_ch = [9, 10, 11, 12, 13, 14, 15, 16], sw_val=[0, 0, 0, 0, 0, 0, 1, 0], dac_ch = [9, 10, 11, 12, 13, 14, 15, 16], dac_val=[355, 18, 798, 1537, 2
869, 866, 3660, 2081]
%lrhs: calibrate weight for input 0
%lrhs: calibrate weight for input 255
%lrhs: top : calibration done
copying files finished
%lrhs: Closing...
[]
```

图 17. 校准程序日志

5. 实际样例

Compass 提供本地和远程两种方式实现服务,

5.1 矩阵点乘（本地执行）

本地调用 Gazelle 指登录到 Gazelle 后，在 Gazelle 机器上运行程序，调用光计算芯片。

1. 从 compass_sdk.fast_calibration.compass_lib 导入 compass_matmul, compass_init
2. 调用 compass_init () 进行初始化
3. 调用 compass_matmul()接口调用光计算芯片实现矩阵乘法。

```
import numpy as np
from compass_sdk.fast_calibration.compass_lib import compass_matmul, compass_init

def test_compass_matmul():
    compass_init(150)
    tia_gain_scale_factor = 256
    for i in range(1):
        vec = np.random.randint(0, 256, (1, 64))
        mac = np.random.randint(-128, 127, (64, 32))
        tomac = compass_matmul(vec, mac)
        treal = np.matmul(vec, mac)
        print('-'*30)
        print(f'vec {vec}\nmac {mac}\nreal {treal}\nmac {tomac * tia_gain_scale_factor}')
        print("tomac = ", tomac)
        print("treal = ", treal)
        print("*****10")
        print("tomac*256 = ", tomac*256)
        print("treal = ", treal)
        print("*****10")
        print("tomac/256 = ", tomac/256)
        print("treal = ", treal)

if __name__ == '__main__':
    test_compass_matmul()
```

图 18. 本地矩阵乘法示例

5.2 矩阵点乘（远程）

Gazelle 支持远程访问，用户程序运行在远程机器上，通过网络发送请求到 Gazelle 上，调度程序实现光计算。

1. 登录 Gazelle，切换到 root 账号下，输入 ifconfig 查询 IP 地址



```
root@bionic-arm64:/home/uisrc/code# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.102.12.18 netmask 255.255.255.0 broadcast 10.102.12.255
    inet6 fe80::17ce:b618:2530:f662 prefixlen 64 scopeid 0x20<link>
    ether 24:86:25:00:00:01 txqueuelen 1000 (Ethernet)
    RX packets 1021574 bytes 142880185 (142.8 MB)
    RX errors 0 dropped 2388 overruns 0 frame 0
    TX packets 1108420 bytes 1301719556 (1.3 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 30

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 271318 bytes 30695707 (30.6 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 271318 bytes 30695707 (30.6 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

图 19. Ifconfig 查询到 IP 地址为 10.102.12.18

2. 切换到/home/uisrc/code 路径，“python3 compass_server.py”启动 server 程序，记录端口号。端口号默认为 8000。

```
root@bionic-arm64:/home/uisrc/code# python3 compass_server.py
open /dev/ttyPS1 serial error
open file success
set chaneell1 laster param: /dev/ttyUSB0 is y = 0.2447x - 5.4282
open file success
%I:rhs: file_name_LUT_typ:doublearm
set chl current 83.48mA
open file success
open file success
%I:rhs: get die_type is die4
%I:rhs: get die_id is N4-D09-1-EVB02-2mA(new)
open file success
open file success
%I:rhs: get die_type is die4
%I:rhs: get die_id is N4-D09-1-EVB02-2mA(new)
open file success
%I:rhs: get vdd is 1.8
%I:rhs: get vee is 0.0
set vee = 0.0,set io value =0
set vdd = 1.8
ltc2264 reg read, data=[255, 135]
ltc2265 verify communication failed,read reg value is:135
ltc2264 reg read, data=[255, 128]
data is :0x80
ltc2264 reg read, data=[255, 203]
data is :0xcb
%I:rhs: get Power_ref =1000, get gc_mpd_offset=2045
read finished flag =3, before update LUT
read finished flag =1, count = 0
read finished flag =3, count = 1
%I:rhs: config tx calibration value: sw_ch = [9, 10, 11, 12, 13, 14, 15, 16],sw_val=[0, 0, 0, 0, 1, 0, 1, 0], dac_ch = [9, 10, 11, 12, 13, 14, 15, 16], dac_val=[251, 0, 910, 1402, 3549, 973, 3744, 2095]
open file success
%I:rhs: get die_type is die4
%I:rhs: get die_id is N4-D09-1-EVB02-2mA(new)
%I:rhs: mg_wgh1_sw_list is [17, 18, 19, 20, 21, 22, 23, 24]
%I:rhs: mg_wgh2_sw_list is [25, 26, 27, 28, 29, 30, 31, 32]
open file success
%I:rhs: get offset value is [2051.29, 2045.07]
0.0.0.0:8000 ready for serve
```

图 20. 启动端口号默认为 8000

3. 拷贝 client 程序到远程机器上，即可在远程机器上远程调用 Gazelle 实现计算，远程调用时，不需要调用 compass_init，因 server 启动时已默认初始化，远程调用时不再需要额外调用 compass_init。Compass_matmulvs 后两个参数填上 IP 和端口号，即可远程调用。

```

1 import os, sys
2 import requests
3 import numpy as np
4 import argparse
5
6 ##### this file run in remote client #####
7 def compass_matmul(vec,mat,host_ip,host_port=8000):
8     # assert datatype
9     response = requests.post(f'http://{host_ip}:{host_port}',
10         json={'act':vec.tolist(), 'weight':mat.tolist()})
11     result = response.json()
12     if 'error' in result:
13         print(result['error'])
14         return None
15     if 'data' in result:
16         return np.array(result['data'])
17
18 def test_compass_matmul():
19
20     vec = np.random.randint(0,256,(16,16))
21     mat = np.random.randint(-128,127,(16,16))
22     res = compass_matmul(vec,mat,'10.102.12.18',8000)
23
24     std_res = np.matmul(vec,mat)
25     print('stand ',std_res)
26     print('remote ',res)
27     pass
28
29 if __name__ == '__main__':
30     test_compass_matmul()

```

图 21. 远程调用实力代码



6.FAQ

1. 远程登录的用户名 uisrc, 密码为 root。
2. 所有代码执行应当在 root 账户下执行, 密码为 root。
3. 代码和 sdk 安装在 python3 环境下, 执行请用 python3。